Architecture Decision Brief: Digital Café Luna

Sergio Alfredo Junca Valero · ST1611 Computación en Nube · EAFIT · Versión final, 23 de octubre de 2026

Resumen. Saco la campaña del cómputo con S3 y CloudFront, atiendo los pedidos con EC2 en Auto Scaling detrás de un ALB en dos AZ, guardo clientes y pedidos en RDS PostgreSQL Multi-AZ y llevo los reportes a una Lambda activada por eventos. Todo se define en CloudFormation y cada alarma tiene una acción o un responsable. Cuesta unos USD 173 en un mes normal y hasta USD 292 en un mes con campaña, a precios de lista. Cada decisión se apoya en un comportamiento que comprobé en los Labs 01 a 07.

1. Problema, restricciones y prioridades

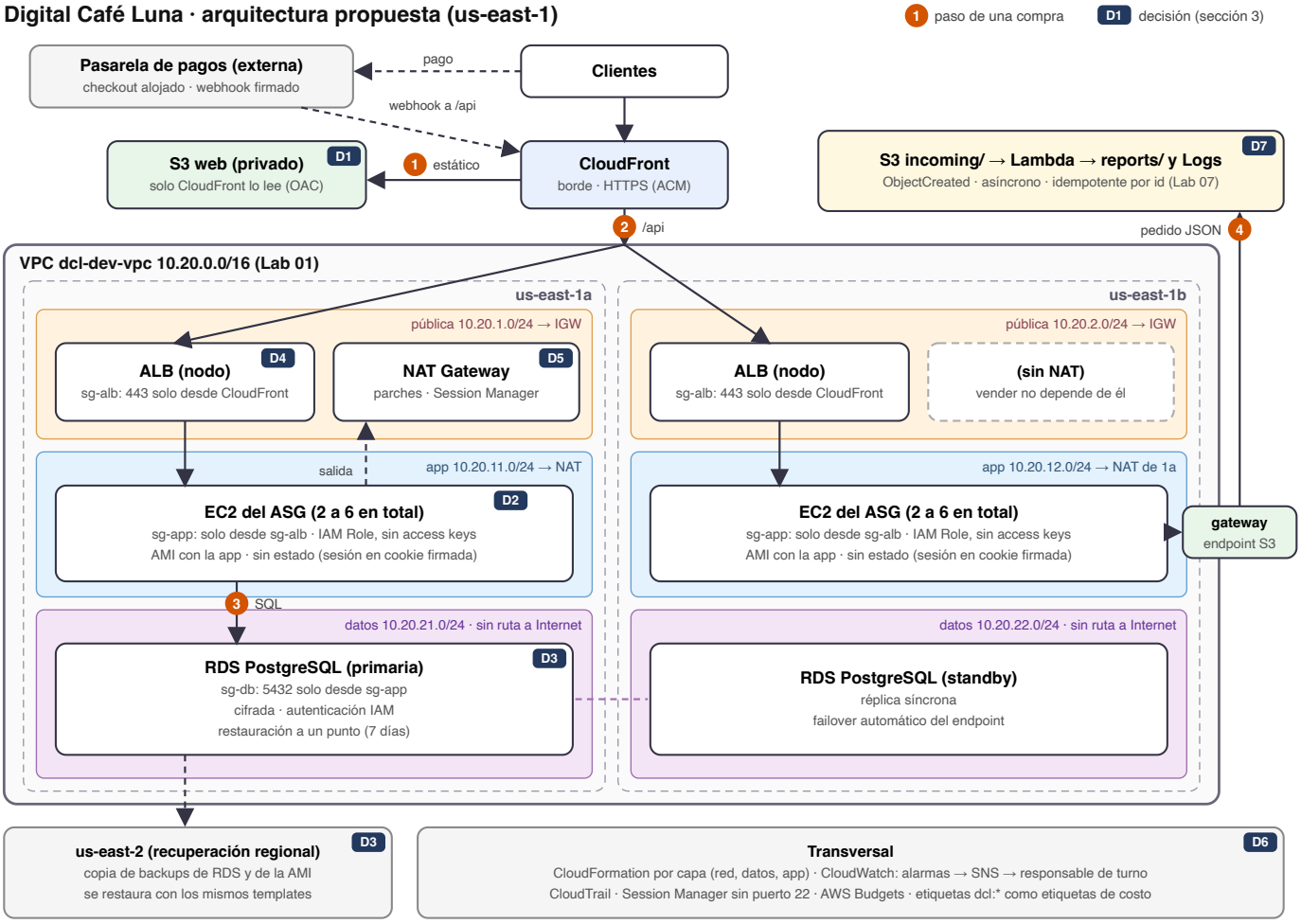
Digital Café Luna vende en sus tiendas y en línea, y prepara una campaña nacional de pocos días con demanda incierta. Hoy su plataforma depende de infraestructura limitada: la capacidad se compra antes de conocer la demanda, así que el pico satura y el promedio desperdicia. El reto es crecer sin aumentar la complejidad operativa.

	Restricción del caso	Qué le exige a la arquitectura
R1	Datos de clientes y pedidos	Un pedido confirmado no se pierde; los datos son privados, cifrados y auditables
R2	Demanda irregular (campaña)	Capacidad que sigue a la demanda, con techo; lo estático, fuera del cómputo
R3	Resiliencia	Seguir vendiendo si falla una instancia o una AZ
R4	Equipo tecnológico pequeño	Operar poco: servicios administrados, todo como código, alarmas con responsable
R5	Costos controlados	Costo fijo mínimo y justificado; costo variable acotado
R6	Capacidad de evolución y nuevas capacidades de negocio	Decisiones reversibles; modernizar por partes

Prioridad cuando chocan: R1 > R2 > R3 > R5 > R6. Perder un pedido confirmado es peor que una página lenta, y el pico de la campaña es más probable que la caída de una AZ. R4 no se negocia: descarto lo que el equipo no pueda operar.

Alcance. Entran la web y la campaña, los pedidos, los clientes y los reportes. El inventario de las tiendas queda para una segunda etapa: en la priorización del caso (S01) tiene valor medio y riesgo alto, frente a la web (valor alto, riesgo medio) y los reportes (riesgo bajo).

2. Arquitectura propuesta



Una compra, paso a paso (los números están en el diagrama):

1. CloudFront entrega la web y la campaña desde su caché. El bucket es privado y solo CloudFront lo lee (Origin Access Control).
2. Las llamadas a `/api` siguen de CloudFront al ALB, que solo acepta tráfico de CloudFront y lo reparte entre las EC2 sanas de las dos AZ.
3. La app guarda el pedido en RDS. El cliente paga en el checkout alojado de la pasarela, y la confirmación vuelve como webhook firmado por `/api`.
4. Con el pedido confirmado, la app deja `incoming/<id>.json` en S3 por el gateway endpoint, y el evento `ObjectCreated` invoca la Lambda de reportes.

Región us-east-1, con los cuatro criterios de S01. Usuarios y latencia: lo estático sale del borde de CloudFront, cerca de Colombia, y la API tolera la ida y vuelta a Virginia. Datos y regulación: supuesto 5. Servicios: todos los que uso están disponibles. Costo y operación: precios entre los más bajos y la región de los labs.

Red: la VPC `10.20.0.0/16` del Lab 01, con tres capas en dos AZ. Las subnets públicas son las del Lab 01. Pública o privada lo decide la route table (Lab 04).

Capa	us-east-1a	us-east-1b	Ruta 0.0.0.0/0
Pública (ALB, NAT)	10.20.1.0/24	10.20.2.0/24	Internet Gateway
App (EC2)	10.20.11.0/24	10.20.12.0/24	NAT Gateway de 1a
Datos (RDS)	10.20.21.0/24	10.20.22.0/24	Ninguna

3. Decisiones

Cada decisión sigue la cadena del curso: restricción, decisión, mecanismo, alternativa descartada y lo que sacrifica. La evidencia está en la sección 7 y el riesgo que queda, en la 4.

D1. Lo estático sale del cómputo: S3 privado y CloudFront. *Restricción:* R2 y R5. *Mecanismo:* el pico de una campaña son páginas e imágenes; CloudFront las responde desde caché y no llegan a las EC2. *Descarté* servirlas desde las EC2: el ASG escalaría por tráfico que no necesita cómputo. *Sacrifico* frescura: cada publicación exige invalidar la caché.

D2. Cómputo: EC2 t3.small en un Auto Scaling Group de 2 a 6 instancias detrás de un ALB, en dos AZ. *Restricción:* R2, R3 y R4 (el equipo ya opera Linux). *Mecanismo:* el health check saca del ALB a la instancia que falla y el ASG repone la capacidad deseada (Lab 02). Uso health check de tipo ELB: en el Lab 02 era de tipo EC2, que solo reemplaza instancias detenidas y no una app colgada. Escala por target tracking (CPU promedio de 50%), y una acción programada sube el mínimo a 4 la víspera de la campaña, porque la fecha se conoce y una instancia nueva tarda minutos en atender. El máximo de 6 es el techo de costo. La instancia arranca desde una AMI con la app instalada y no guarda estado: la sesión viaja en una cookie firmada, el carrito vive en la base y los archivos en S3. *Descarté* la capacidad fija para el pico (USD 61 más al mes) y serverless para todo (rediseño y competencias nuevas antes de la campaña). *Sacrifico* elasticidad fina: fuera de campaña quedan 2 instancias aunque sobre capacidad.

D3. Clientes y pedidos en RDS PostgreSQL Multi-AZ. *Restricción:* R1, R3 y R4. *Mecanismo:* cada escritura se replica de forma síncrona a un standby en la otra AZ y, si falla la primaria, el endpoint pasa al standby sin cambiar la app. AWS opera parches y backups: restauración a cualquier punto de los últimos 7 días y copia de los backups a us-east-2. El versionado de S3 no reemplaza este respaldo: protege objetos, no la base. *Descarté* PostgreSQL en EC2 (el equipo operaría réplica, backups y parches) y DynamoDB (escala más, pero obliga a remodelar por patrones de acceso y ata más a AWS). *Sacrifico* USD 26 al mes frente a Single-AZ.

D4. Una sola puerta de entrada y controles por capa, con la cadena de S05: red, identidad, cifrado y trazabilidad. *Restricción:* R1.

- *Red:* sg-alb acepta 443 solo desde la prefix list administrada de CloudFront, así que nadie llega directo al ALB. sg-app acepta solo desde sg-alb, y sg-db acepta 5432 solo desde sg-app. Las reglas referencian SG, no IP (Lab 04). La app y la base no tienen IP pública. CloudFront incluye Shield Standard contra DDoS sin costo.
- *Identidad:* IAM Role en las EC2 y en la Lambda, sin access keys, con Action y Resource mínimos (Lab 05). La app entra a PostgreSQL con autenticación IAM: un token de 15 minutos firmado con su role, sin contraseña guardada. RDS administra la contraseña maestra en Secrets Manager. El operador entra por Session Manager, sin puerto 22 ni bastión.
- *Cifrado:* SSE-S3 en los buckets, RDS cifrado en reposo con KMS y TLS con certificados de ACM.
- *Trazabilidad:* CloudTrail registra quién cambió IAM, un SG o la base (Lab 05).

Descarté un ALB abierto a Internet. *Sacrifico* ritmo de conexiones: la autenticación IAM limita las conexiones nuevas por segundo, lo que se resuelve con un pool de conexiones por instancia.

D5. Salida a Internet: gateway endpoint para S3 y un solo NAT, porque vender no depende de él. *Restricción:* R5 frente a R3. *Mecanismo:* el tráfico a S3 (pedidos, reportes y configuración) va por el gateway endpoint, sin costo y sin pasar por el NAT. Por el NAT solo pasa lo que no detiene la venta: parches, Session Manager y logs del agente. La instancia arranca de una AMI preparada, el token de la base se firma localmente y el pago entra como webhook. *Descarté* un NAT por AZ (USD 37 más al mes). *Sacrifico* la salida redundante: si cae la AZ del NAT, las instancias de la otra AZ siguen vendiendo, pero sin parches ni Session Manager hasta que CloudFormation lo recree.

D6. Operación como código y alarmas con responsable. *Restricción:* R4. *Mecanismo:* un template de CloudFormation por capa (red, datos, app) crea, actualiza con change set y borra (Lab 06). El mismo template levanta un ambiente temporal para la prueba de carga y lo borra al terminar. Una alarma que nadie revisa es solo una notificación: en el Lab 06 la alarma tenía `ActionsEnabled: false`, y aquí cada alarma actúa. El ASG reacciona solo, con target tracking. Las alarmas que piden criterio humano (5XX y latencia del ALB, menos de 2 targets sanos, CPU, créditos y conexiones de RDS, `Errors` de la Lambda y presupuesto) avisan por SNS al responsable de turno, que sigue un runbook de una página. Las etiquetas `dcl:project` y `dcl:environment` de los labs se activan como etiquetas de costo. *Descarté* Terraform: es portable, pero obliga a custodiar el archivo de estado, y con una sola nube y un equipo pequeño CloudFormation guarda el estado por nosotros. *Sacrifico* portabilidad de la definición de infraestructura.

D7. Modernización parcial: reportes por evento con Lambda. *Restricción:* R4 y R2; los reportes son el workload de menor riesgo del caso. *Mecanismo:* la app escribe `incoming/<id>.json`, y S3 emite `ObjectCreated` (filtro `incoming/` y `.json`) e invoca la Lambda de forma asíncrona. La Lambda escribe `reports/<fecha>/<id>.json` y una línea JSON en CloudWatch Logs con el mismo id (Lab 07). S3 entrega el evento al menos una vez; como la salida usa el id del pedido, un evento repetido

sobrescribe el mismo objeto y no duplica. El filtro de prefijo evita que la escritura en `reports/` vuelva a invocar la función: en el Lab 07, `archive/` no invocó. *Descarté* un proceso en las EC2 (compite con la API en el pico) y una VM dedicada (un servidor encendido para un trabajo intermitente). *Sacrifico* inmediatez y orden de los eventos. Si un evento falla tras los reintentos, el reporte se reprocesa desde `incoming/`, y el pedido sigue en RDS.

D8. Lo que no hago ahora: contenedores, serverless para el núcleo de pedidos, NoSQL y multi-región activa. Cada uno resuelve un problema que el caso todavía no tiene y exige operación que el equipo no tiene (R4). La sección 6 dice qué señal haría cambiar cada uno.

4. Trade-offs, riesgos y supuestos

Decisión	Gana	Sacrifica	Riesgo que conserva	Cómo lo detecto
D1 CloudFront	El pico estático no toca el cómputo	Frescura inmediata	Contenido viejo tras publicar	Revisión después de cada invalidación
D2 ASG de 2 a 6	Capacidad que sigue la demanda, con techo	Capacidad por encima de 6	Un pico mayor degrada la API	Latencia p95 alta con el ASG en su máximo
D3 RDS Multi-AZ	Failover sin perder pedidos	USD 26 al mes	La base es el cuello de botella del pico	Alarmas de CPU, créditos y conexiones de RDS
D4 Prefix list e IAM	Una sola puerta; sin llaves ni contraseñas en la app	Conexiones nuevas por segundo	Una policy mal escrita	AccessDenied en los logs de la app
D5 Un solo NAT	USD 37 al mes	Salida redundante	Sin parches ni Session Manager si cae su AZ	Evento de AWS Health en la AZ del NAT
D6 CloudFormation	Cambios repetibles y auditables	Portabilidad	Cambios manuales fuera del template	Detección de drift de CloudFormation
D7 Lambda	Reportes sin servidor que operar	Inmediatez y orden	Evento fallido tras los reintentos	Alarma sobre `Errors`
D8 Sin multi-región	No duplicar el costo fijo	RTO regional de horas	Caída larga si falla la región	Ensayo de restauración en us-east-2

Resiliencia por escenario (los objetivos son el supuesto 3):

Falla	Qué ocurre	RTO / RPO	Cómo lo pruebo
Instancia o app colgada	El ALB la saca y el ASG la reemplaza	Segundos / 0	Terminar una instancia y detener la app en otra
Una AZ completa	ALB y ASG siguen en la otra AZ; RDS pasa al standby	1 a 2 min / 0	Reboot con failover de RDS durante la prueba de carga
Error humano o dato corrupto	RDS se restaura a un punto en el tiempo; el bucket de reportes guarda versiones (Lab 03)	1 h / 5 min	Restaurar a un punto anterior en el ambiente temporal
La región	Restaurar la copia de backups y los templates en us-east-2	4 h / 1 h	Ensayo de restauración completa

Supuestos, cada uno con su validación:

1. El pico es unas 10 veces la carga normal durante 3 a 5 días, y la mayoría de las peticiones son contenido estático. *Se valida* con la prueba de carga y el cache hit ratio de CloudFront.
2. La app puede ser stateless: sesión en cookie firmada, carrito en la base y archivos en S3. *Se valida* terminando una instancia con usuarios activos sin perder el carrito.
3. El negocio acepta 99,9% de disponibilidad en campaña, p95 de la API bajo 1 s y los RTO/RPO de la tabla. *Se valida* con el negocio, y se mide con métricas del ALB y de RDS.
4. El pago usa el checkout alojado de una pasarela: la plataforma no guarda tarjetas ni llama a la pasarela. *Se valida* con el proveedor. Si exige llamadas salientes, la salida pasa a ser crítica y pongo un NAT por AZ (USD 37 más al mes).
5. Los datos personales pueden alojarse en EE. UU.: la Ley 1581 de 2012 permite transferirlos a países con nivel adecuado de protección. *Se valida* con el área legal.
6. El equipo domina Linux y SQL, no contenedores. *Se valida* con el equipo.
7. El presupuesto tolera unos USD 175 en un mes normal y USD 300 en un mes de campaña; el caso no da cifra. *Se valida* con AWS Budgets y una alerta al 80%.

5. Costo aproximado (USD al mes, us-east-1)

Precios de lista on-demand de la AWS Price List API, la misma fuente de la Pricing Calculator, consultados el 21 y el 23 de octubre de 2026. CloudFront con la tarifa de Sudamérica; meses de 730 horas. No descuento capas gratuitas. Consumo supuesto:

- **Mes normal:** 2 EC2, 1 LCU, 50 GB por el NAT, 100 GB y 2 millones de peticiones en CloudFront, 30.000 pedidos.
- **Mes con campaña:** además, 5 días (120 h) con 6 EC2 al 50% de CPU, 10 LCU, 600 GB y 9 millones de peticiones, y 150.000 pedidos.

Componente	Precio de lista	Normal	Campaña
RDS PostgreSQL Multi-AZ db.t4g.small, 20 GB gp3	USD 0,065/h; 0,23/GB-mes; créditos de CPU a 0,075/vCPU-h	52	56
NAT Gateway	USD 0,045/h; 0,045/GB procesado	35	36
EC2 t3.small, 8 GB gp3 cada una	USD 0,0208/h; 0,08/GB-mes	32	42
Créditos de CPU de t3 (modo unlimited, sobre la línea base de 20%)	USD 0,05/vCPU-h	0	22
ALB	USD 0,0225/h; 0,008/LCU-h	22	31
CloudFront (Sudamérica) y S3	USD 0,110/GB; 0,022 por 10.000 peticiones HTTPS; S3 a 0,023/GB-mes	16	88
IPv4 públicas (2 del ALB, 1 del NAT)	USD 0,005/h cada una	11	11
Copia de backups en us-east-2	USD 0,095/GB-mes; 0,01/GB transferido	2	2
CloudWatch, Lambda, Secrets Manager y tráfico entre AZ	Alarma 0,10/mes; logs 0,50/GB; Lambda 0,20 por millón; secreto 0,40/mes	3	4
Total		**173**	**292**

- **Cost drivers.** El 85% del mes normal es fijo: RDS, NAT, horas del ALB, IPv4 y las 2 EC2 base. La elasticidad ahorra en el pico, pero el piso lo ponen la base de datos y la red. En campaña manda el tráfico: CloudFront (USD 88) y los créditos de CPU (USD 22) pesan más que las EC2 extra (USD 10). Las t3 son burstable, y por encima de su línea base el modo unlimited cobra el excedente.
- **Palancas.** La capa gratuita permanente de CloudFront (1 TB y 10 millones de peticiones al mes) cubre ambos escenarios y deja los totales en USD 158 y 206. El precio reservado de RDS espera a que la carga base sea estable, tras 3 meses de métricas. El ambiente de pruebas existe solo durante la prueba. Dejar 6 EC2 fijas todo el mes costaría USD 61 más: por eso escalo en vez de dimensionar para el pico.
- **Gobierno del costo.** AWS Budgets con alerta al 80% (supuesto 7) y etiquetas de costo por ambiente (D6).

6. Evolución: qué modernizar ahora y qué después

Cuándo	Qué	Señal que lo dispara
Ahora	Web y campaña (D1), pedidos (D2, D3) y reportes por evento (D7)	Es lo que la campaña necesita
Después de la campaña	Inventario de tiendas: cada tienda sube su archivo a S3 y se procesa con el patrón de eventos de D7	Campaña cerrada y reportes estables
Siguiente	Destino on-failure para la Lambda; WAF en CloudFront	`Errors` mayor que 0 en la Lambda; bots o abuso en los logs
Siguiente	ElastiCache para el catálogo	CPU de RDS sobre 60% o p95 sobre 1 s por lecturas
Después	ECS Fargate para la API	Despliegues semanales y un equipo formado en contenedores
Después	Warm standby en us-east-2	El negocio exige un RTO regional menor a 1 h y acepta casi duplicar el costo fijo

Capacidades nuevas. Fidelización o analítica leen los reportes que ya quedan en S3, sin cargar la base ni tocar la API.

Lock-in consciente. PostgreSQL, Linux y HTTP son portables. Lambda y CloudFormation atan a AWS, pero en piezas pequeñas: una función de reportes y unos templates que, además, documentan la arquitectura.

7. Evidencia técnica propia: Labs 01 a 07

Corrí los siete labs en mi cuenta, con cleanup verificado. Los resultados están en https://sjunka.github.io/ComputacionEnNube/.

Lab	Qué comprobé	Sustenta
01 Red base	VPC `dcl-dev-vpc` 10.20.0.0/16 con 10.20.1.0/24 en 1a y 10.20.2.0/24 en 1b. Dos AZ preparan la alta disponibilidad, pero no la dan sin cómputo repartido	Red
02 Resiliencia	El ALB repartió 10/10 entre dos AZ. Terminé una instancia, salió del target group y el ASG la reemplazó. Con health check de tipo EC2, una app colgada no se habría reemplazado	D2
03 Almacenamiento	S3 guardó versiones de un objeto, y EFS dio el mismo archivo a dos instancias en AZ distintas. Object, block y file no son intercambiables	D2, D3
04 Networking	Subnet pública solo por su route table. PASS, FAIL (timeout) y PASS de A a B:8080 al quitar y restaurar una regla de SG que referencia a otro SG	Red, D4
05 Seguridad	Mismo role, misma red: GetObject permitido en `allowed/*`, AccessDenied en otro Resource y en PutObject. SSE-S3, y AttachRolePolicy visible en CloudTrail	D4
06 Operación	Stack en CREATE_COMPLETE en 26 s. Alarma con Average ≥ 70% en 2 de 2 periodos de 5 min: OK, ALARM con CPU de 75, 100 y 100%, y OK al terminar. Update v1 a v2 sin reemplazo y Delete sin residuos. La alarma no tenía acciones	D6
07 Modernización	`incoming/order-001.json` produjo `ObjectCreated:Put`, una invocación y una línea en CloudWatch Logs con el mismo key y requestId. Sin permiso de invocación, S3 rechazó la notificación; `.txt` y `archive/` no invocaron	D7

Lo que los labs no cubren: CloudFront, RDS Multi-AZ, el NAT y el comportamiento bajo carga real. Para eso están las pruebas de la sección 8.

8. Cómo validaría el comportamiento

	Prueba	Evidencia esperada	Valida
1	Prueba de carga hasta 10 veces la carga normal, en un ambiente temporal	El ASG no pasa de 6; p95 bajo 1 s; cache hit alto en CloudFront	D1, D2, supuesto 1
2	Terminar una instancia y detener la app en otra	El ALB saca ambas y el ASG reemplaza las dos	D2
3	Reboot con failover de RDS durante la carga	La app vuelve a escribir en 1 a 2 min; los pedidos confirmados coinciden antes y después	D3
4	Llamar al ALB directo y conectarse a la base desde un SG ajeno	Timeout en ambos; por CloudFront responde 200	D4
5	Con el role de la app, borrar en `reports/`; cambiar un SG con otro usuario	AccessDenied en lo primero; CloudTrail muestra quién y cuándo cambió el SG	D4
6	Quitar la ruta al NAT en una subnet de app	La venta sigue (pedido y webhook); solo fallan los parches	D5
7	Subir dos veces el mismo pedido a `incoming/`, y un `.txt`	Dos invocaciones y un solo objeto en `reports/`; el `.txt` no invoca	D7
8	Restaurar la base a un punto en el tiempo y el stack completo en us-east-2	RTO medido frente a 1 h y 4 h	D3, D6